Eryk Bukowski 266096

## 1. Cel projektu

Celem projektu jest zaprojektowanie, zaimplementowanie oraz przetestowanie sieci neuronowej zdolnej do skutecznego rozpoznawania znaków drogowych na podstawie obrazów. System ma na celu klasyfikację różnych typów znaków drogowych, co może znaleźć zastosowanie w systemach wspomagania kierowcy oraz autonomicznych pojazdach.

## 2. Baza danych

Do projektu wykorzystano bazę danych „Polish Traffic Signs Dataset" pobrana ze strony www.kaggle.com. Baza pierwotnie zawierała 20 rodzajów polskich znaków drogowych, jednak na potrzeby projektu klasy zbyt mało reprezentowane zostały usunięte. Ostatecznie w bazie danych pozostało 15 rodzajów znaków. Dla pozostałych klas, które zawierały mniej niż 200 przykładów, zastosowano losową augmentację danych w celu wyrównania liczebności wszystkich klas. Proces augmentacji obejmował: rozjaśnienie, przyciemnienie, obrót, pochylenie lub rozmycie obrazu. Obrazy z każdej klasy zostały podzielone na zbiory: treningowy, walidacyjny i testowy, w stosunku 70:15:15.




Rys 1. Prezentacja przykładowych elementów bazy danych

## 3. Wykorzystany model

Do realizacji projektu wykorzystano model konwolucyjnej sieci neuronowej. Architektura sieci została dostosowana do danych wejściowych w postaci kolorowych obrazów. Model CNN składa się z naprzemiennie występujących warstw:

- Warstwa wejściowa – Warstwa określająca rozmiar obrazu. Liczby odpowiadają kolejno wysokości, szerokości oraz liczbie kanałów. W przypadku obrazów kolorowych liczba kanałów ustawiona została na 3.
- Warstwa konwolucyjna – Składa się z zestawu filtrów w formie niewielkich macierzy. Filtry przesuwają się po obrazie wykonując operację konwolucji i generując mapę cech. Warstwa ma za zadanie wykrywanie i wydobywanie istotnych cech z obrazów takich jak np. krawędzie, wzory.
- Warstwa normalizacji – Warstwa normalizuje rozkład danych wewnątrz sieci, a następnie umożliwia sieci uczenie się odpowiedniego skalowania i przesunięcia aktywacji. Warstwa poprawia efektywność i stabilność treningu sieci.
- Warstwa ReLU – Funkcja aktywacji, która odcina wartości ujemne i przepuszcza dodatnie, co wprowadza nieliniowość do modelu.
- Warstwa pooling – Redukuje rozmiar obrazów, zachowujące najważniejsze informacje. Max pooling wybiera największą wartość z danego okna. Dzięki warstwie sieć staje się bardziej wydajna i łatwiejsza w uczeniu.
- Warstwa w pełni połączona – Warstwa łączy ze sobą wszystkie cechy wykryte przez poprzednie warstwy.
- Warstwa Softmax – Konwertuje surowe wyjścia sieci neuronowej na rozkład prawdopodobieństwa klas. Dzięki tej warstwie model może wskazać, do której klasy najbardziej pasuje dane wejście.
- Warstwa klasyfikacji – Ostatni element sieci, który dokonuje przypisania danych do konkretnej klasy.

```
layers = [
    imageInputLayer([16 16 3])

    convolution2dLayer(3,8,'Padding','same')
    batchNormalizationLayer
    reluLayer

    maxPooling2dLayer(2,'Stride',2)

    convolution2dLayer(3,16,'Padding','same')
    batchNormalizationLayer
    reluLayer

    maxPooling2dLayer(2,'Stride',2)

    convolution2dLayer(3,32,'Padding','same')
    batchNormalizationLayer
    reluLayer

    maxPooling2dLayer(2,'Stride',2)

    fullyConnectedLayer(15)
    softmaxLayer
    classificationLayer];
```

Rys 2. Prezentacja architektury przykładowego modelu CNN

W procesie uczenia sieci zastosowano funkcję early stop oraz następujące parametry treningowe:

- Initial learn rate – określa początkową długość kroków wykonywanych podczas aktualizacji wag w trakcie treningu. Zbyt duży learning rate może prowadzić do niestabilnego uczenia, a zbyt mały do wolnej zbieżności.
- Max epochs – określa maksymalną liczbę pełnych przejść przez cały zestaw danych treningowych. Po przekroczeniu tej liczby trening zostaje zatrzymany.
- Validation frequency – określa co ile iteracji wykonywana jest walidacja.
- Loss Threshold – określa minimalną wartość funkcji straty. Gdy wartość funkcji spadnie poniżej tej wartości funkcja early stop zatrzymuje proces uczenia.

```
lossThreshold = 0.01;
options = trainingOptions('sgdm', ...
    'InitialLearnRate',0.01, ...
    'MaxEpochs',50, ...
    'Shuffle','every-epoch', ...
    'ValidationData',imdsValidation, ...
    'ValidationFrequency',2, ...
    'Verbose',false, ...
    'Plots','training-progress', ...
    'OutputFcn', @(info)stopTraining(info, lossThreshold));
```

Rys 3. Parametry treningowe

## 4. Przeprowadzone eksperymenty

W celu określenia optymalnych parametrów modelu oraz uzyskania jak najwyższej skuteczności klasyfikacji przeprowadzono szereg eksperymentów mających na celu zbadanie wpływu różnych parametrów modelu lub bazy danych na efektywność działania sieci. Do oceny efektywności zostały wykorzystane następujące metryki:

- Precyzja – Określa jak wiele z przypisanych do danej klasy przypadków zostało przypisanych poprawnie.
- Czułość – Określa jak wiele ze wszystkich pozytywnych przypadków zostało sklasyfikowane jako pozytywne.
- Swoistość - Określa jak wiele ze wszystkich negatywnych przypadków zostało sklasyfikowane jako negatywne.
- F1 – Miara ta jest średnią harmoniczną precyzji i czułości.
- Dokładność – Jest stosunkiem poprawnych do wszystkich klasyfikacji.

### 4.1 Rozdzielczość obrazów

Oryginalne obrazy w bazie danych miały różne rozdzielczości i zostały poddane procesowi skalowania w celu ujednolicenia ich wymiarów na potrzeby testów. W tym celu wykorzystana została funkcja imresize dostępna w programie Matlab. Eksperyment przeprowadzono zaczynając od rozdzielczości 128 na 128 pikseli a następnie stopniowo zmniejszano wymiary obrazów aż do wymiarów 8 na 8 pikseli.

Tabela 1. Wpływ rozdzielczości zdjęć na efektywność modelu

| Rozdzielczość | Czas treningu | liczba epok | liczba iteracji | metryki | średnia |
|---|---|---|---|---|---|
| 128x128 | 35s | 5 | 65 | Precyzja | 97,74% |
| | | | | Czułość | 97,56% |
| | | | | Swoistość | 99,83% |
| | | | | F1 | 97,65% |
| | | | | Dokładność | 97,56% |
| 64x64 | 22s | 9 | 140 | Precyzja | 99,78% |
| | | | | Czułość | 99,78% |
| | | | | Swoistość | 99,98% |
| | | | | F1 | 99,78% |
| | | | | Dokładność | 99,78% |
| 32x32 | 11s | 10 | 160 | Precyzja | 99,35% |
| | | | | Czułość | 99,33% |
| | | | | Swoistość | 99,95% |
| | | | | F1 | 99,34% |
| | | | | Dokładność | 99,34% |
| 16x16 | 7s | 10 | 160 | Precyzja | 99,13% |
| | | | | Czułość | 99,11% |
| | | | | Swoistość | 99,94% |
| | | | | F1 | 99,12% |
| | | | | Dokładność | 99,11% |
| 8x8 | 5s | 10 | 160 | Precyzja | 96,05% |
| | | | | Czułość | 96,00% |
| | | | | Swoistość | 99,71% |
| | | | | F1 | 96,02% |
| | | | | Dokładność | 96,00% |

Dla wszystkich testowanych rozdzielczości obrazów sieć osiągnęła bardzo dobre rezultaty. Znaczne różnice zaobserwowano natomiast w czasie treningu oraz w liczbie wymaganych iteracji. Obniżenie rozdzielczości obrazów skutkowało znacznym skróceniem czasu trenowania modelu, jednak wymagało większej liczby iteracji. Dla wymiarów mniejszych niż 32x32 sieć wykorzystała maksymalną ustaloną liczbę 10 epok. Do dalszych testów przyjęto rozdzielczość równą 16 na 16 pikseli.

### 4.2 Liczba warstw

Eksperyment przeprowadzono dla różnych konfiguracji par warstw składających się z warstwy konwolucyjnej i poolingowej. Liczbę takich par podczas eksperymentu zwiększano od 2 do 5, w celu zbadania wpływu rozbudowania modelu na jego skuteczność.

Tabela 2. Wpływ liczby warstw na efektywność modelu

| warstwy | czas treningu | liczba epok | liczba iteracji | metryki | średnia |
|---|---|---|---|---|---|
| 3 | 10s | 17 | 268 | Precyzja | 99,57% |
| | | | | Czułość | 99,56% |
| | | | | Swoistość | 99,97% |
| | | | | F1 | 99,56% |
| | | | | Dokładność | 99,56% |
| 2 | 7s | 14 | 215 | Precyzja | 98,52% |
| | | | | Czułość | 98,44% |
| | | | | Swoistość | 99,89% |
| | | | | F1 | 98,48% |
| | | | | Dokładność | 98,44% |
| 4 | 12s | 19 | 290 | Precyzja | 98,94% |
| | | | | Czułość | 98,89% |
| | | | | Swoistość | 99,92% |
| | | | | F1 | 98,81% |
| | | | | Dokładność | 98,89% |
| 5 | 19s | 29 | 451 | Precyzja | 98,93% |
| | | | | Czułość | 98,89% |
| | | | | Swoistość | 99,92% |
| | | | | F1 | 98,91% |
| | | | | Dokładność | 98,89% |

Wyniki uzyskane dla wszystkich testowanych konfiguracji były bardzo dobre i wykazywały niewielkie różnice pod względem dokładności. Zwiększanie liczby warstw skutkowało wydłużeniem czasu treningu oraz wzrostem liczby wymaganych iteracji. Najbardziej optymalna wydaje się być konfiguracja z trzema warstwami, która uzyskała najwyższą efektywność przy relatywnie krótkim czasie treningu.

### 4.3 Algorytm optymalizacji

Eksperyment przeprowadzono z wykorzystaniem trzech różnych algorytmów optymalizacji: Stochastic gradient descent with momentum (SGDM), Adaptive moment estimation (Adam) oraz Root mean square propagation (RMSProp). Celem analizy było porównanie ich wpływu na jakość wyników modelu oraz skuteczność treningu.

- SGDM – Stanowi rozszerzenie algorytmu SGD poprzez wprowadzenie mechanizmu pędu. Polega na uwzględnieniu poprzednich aktualizacji wag w celu przyspieszenia uczenia.
- RMSProp – Algorytm adaptacyjny, który reguluje współczynnik uczenia poprzez normalizację gradientu za pomocą średniej kwadratowej.
- Adam – Łączy zalety algorytmów wykorzystujących momentum i adaptacyjne współczynniki uczenia. Algorytm oblicza pierwszy i drugi moment gradientów, które są odpowiednio oszacowaniami średniej i niecentrowanej wariancji gradientów. Te momenty są następnie wykorzystywane do aktualizacji parametrów modelu.

Tabela 3. Wpływ algorytmu optymalizacji na efektywność modelu

| solver | czas treningu | liczba epok | liczba iteracji | metryki | średnia |
|---|---|---|---|---|---|
| sgdm | 10s | 17 | 268 | Precyzja | 99,57% |
| | | | | Czułość | 99,56% |
| | | | | Swoistość | 99,97% |
| | | | | F1 | 99,56% |
| | | | | Dokładność | 99,56% |
| adam | 3s | 4 | 53 | Precyzja | 97,83% |
| | | | | Czułość | 97,78% |
| | | | | Swoistość | 99,84% |
| | | | | F1 | 97,80% |
| | | | | Dokładność | 97,78% |
| rmsprop | 4s | 5 | 79 | Precyzja | 98,74% |
| | | | | Czułość | 98,67% |
| | | | | Swoistość | 99,90% |
| | | | | F1 | 98,70% |
| | | | | Dokładność | 98,67% |

Najlepszą dokładność uzyskano po zastosowaniu solvera SGDM, jednak wiązało się to z ponad dwukrotnie dłuższym czasem treningu oraz trzykrotnie większą liczbą epok w porównaniu do pozostałych algorytmów. Jako najbardziej optymalny wybrano solver RMSProp, który osiągnął zbliżoną dokładność przy znacznie krótszym czasie treningu.

### 4.4 Szum

Celem eksperymentu była analiza wpływu zaszumienia obrazów na efektywność modelu. Do przeprowadzenia testów wykorzystany został szum Gaussa, który został dodany do obrazów za pomocą funkcji imnoise dostępnej w programie Matlab. Testy przeprowadzono na modelu składającym się z trzech warstw i trenowanym z wykorzystaniem solvera SGDM.

Tabela 4. Wpływ zaszumienia obrazów na efektywność modelu

| Szum | czas treningu | liczba epok | liczba iteracji | metryki | średnia |
|---|---|---|---|---|---|
| 0% | 10s | 17 | 268 | Precyzja | 99,57% |
| | | | | Czułość | 99,56% |
| | | | | Swoistość | 99,97% |
| | | | | F1 | 99,56% |
| | | | | Dokładność | 99,56% |
| 1% | 12s | 23 | 362 | Precyzja | 98,28% |
| | | | | Czułość | 98,22% |
| | | | | Swoistość | 99,87% |
| | | | | F1 | 98,25% |
| | | | | Dokładność | 98,22% |
| 5% | 14s | 28 | 440 | Precyzja | 94,90% |
| | | | | Czułość | 94,67% |
| | | | | Swoistość | 99,62% |
| | | | | F1 | 94,78% |
| | | | | Dokładność | 94,67% |
| 10% | 22s | 45 | 717 | Precyzja | 88,53% |
| | | | | Czułość | 88,44% |
| | | | | Swoistość | 99,17% |
| | | | | F1 | 88,49% |
| | | | | Dokładność | 88,44% |
| 15% | 32s | 49 | 771 | Precyzja | 82,83% |
| | | | | Czułość | 82,22% |
| | | | | Swoistość | 98,73% |
| | | | | F1 | 82,52% |
| | | | | Dokładność | 82,22% |
| 20% | 27s | 50 | 800 | Precyzja | 81,73% |
| | | | | Czułość | 81,33% |
| | | | | Swoistość | 98,67% |
| | | | | F1 | 81,53% |
| | | | | Dokładność | 81,33% |

Efektywność działania sieci, zgodnie z oczekiwaniami spada wraz ze wzrostem poziomu zaszumienia danych wejściowych. Zwiększenie szumu skutkowało wydłużeniem czas treningu oraz wzrostem liczby wymaganych iteracji. Pomimo wysokiego poziomu zaszumienia sięgającego 20%, model uzyskał średnią dokładność na poziomie 81,33% co stanowi zadowalający wynik. W warunkach dużego zaszumienia model poradził sobie najgorzej dla znaków: A17, A21 oraz B33, gdzie precyzja spadła poniżej 70%. Mimo to, nawet przy znacznym zaszumieniu, sieć wykazała bardzo wysoką średnią swoistość, co oznacza dobrą zdolność do prawidłowego odrzucania błędnych klas.

### 4.5 Liczba warstw przy dużym zaszumieniu

Eksperyment miał na celu ponowne zbadanie wpływu liczby warstw jednak teraz przy znacznym zaszumieniu danych wejściowych.

Tabela 5. Wpływ liczby warstw przy znacznym zaszumieniu

| Warunki | czas treningu | liczba epok | liczba iteracji | metryki | średnia |
|---|---|---|---|---|---|
| 20%, 3 warstwy | 27s | 50 | 800 | Precyzja | 81,73% |
| | | | | Czułość | 81,33% |
| | | | | Swoistość | 98,67% |
| | | | | F1 | 81,53% |
| | | | | Dokładność | 81,33% |
| 20%, 2 warstwy | 22s | 49 | 769 | Precyzja | 85,07% |
| | | | | Czułość | 84,00% |
| | | | | Swoistość | 98,86% |
| | | | | F1 | 84,53% |
| | | | | Dokładność | 84,00% |
| 20%, 4 warstwy | 24s | 40 | 625 | Precyzja | 76,68% |
| | | | | Czułość | 76,22% |
| | | | | Swoistość | 98,30% |
| | | | | F1 | 76,45% |
| | | | | Dokładność | 76,22% |
| 20%, 5 warstwy | 29s | 41 | 656 | Precyzja | 78,58% |
| | | | | Czułość | 77,78% |
| | | | | Swoistość | 98,41% |
| | | | | F1 | 78,18% |
| | | | | Dokładność | 77,78% |

Przy wysokim zaszumieniu danych najlepsze wyniki osiągnął model o najmniejszej liczbie warstw. Charakteryzował się on najwyższą dokładnością przy najkrótszym czasie treningu. Wraz ze wzrostem liczby warstw obserwowano spadek efektywności modelu. Analogicznie dla testów ze zmiennym poziomem

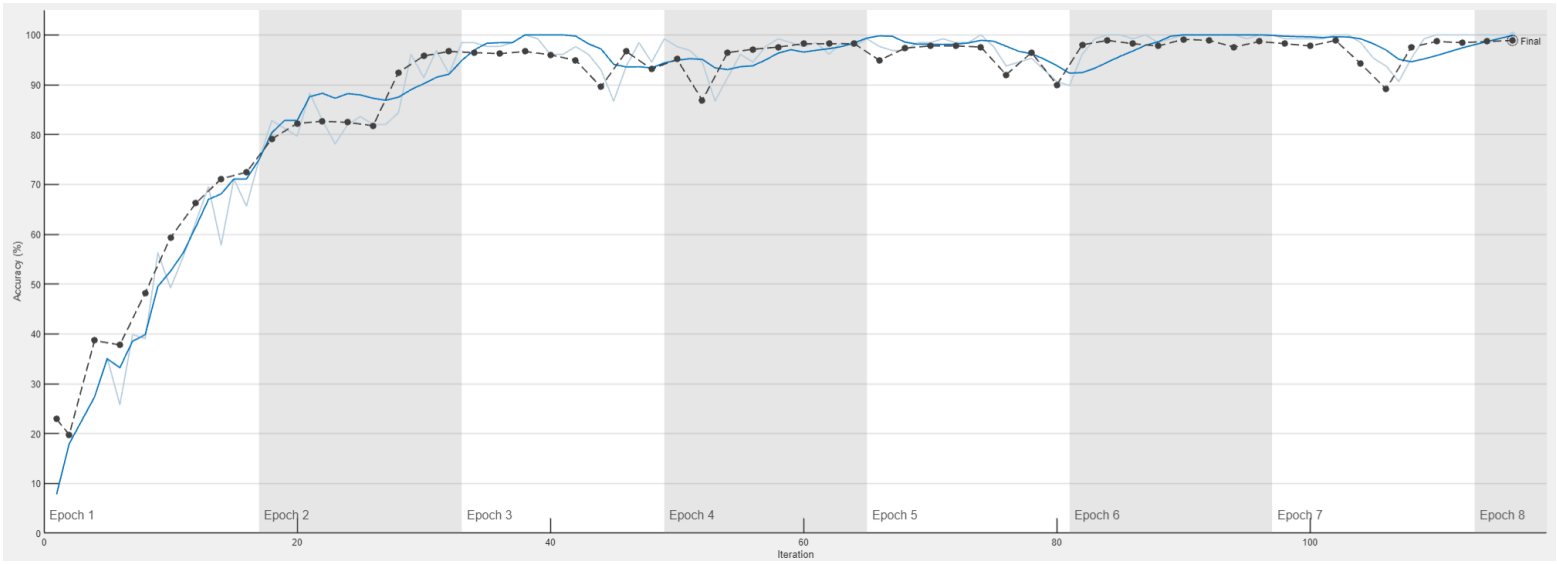
zaszumienia, pomimo spadającej dokładności model utrzymywał bardzo wysoką wartość swoistości. Eksperyment pozwala zauważyć, że mniej złożone modele z mniejszą liczbą warstw są bardziej odporne na zakłócenia w danych wejściowych.

## 5. Wybór najbardziej optymalnego modelu

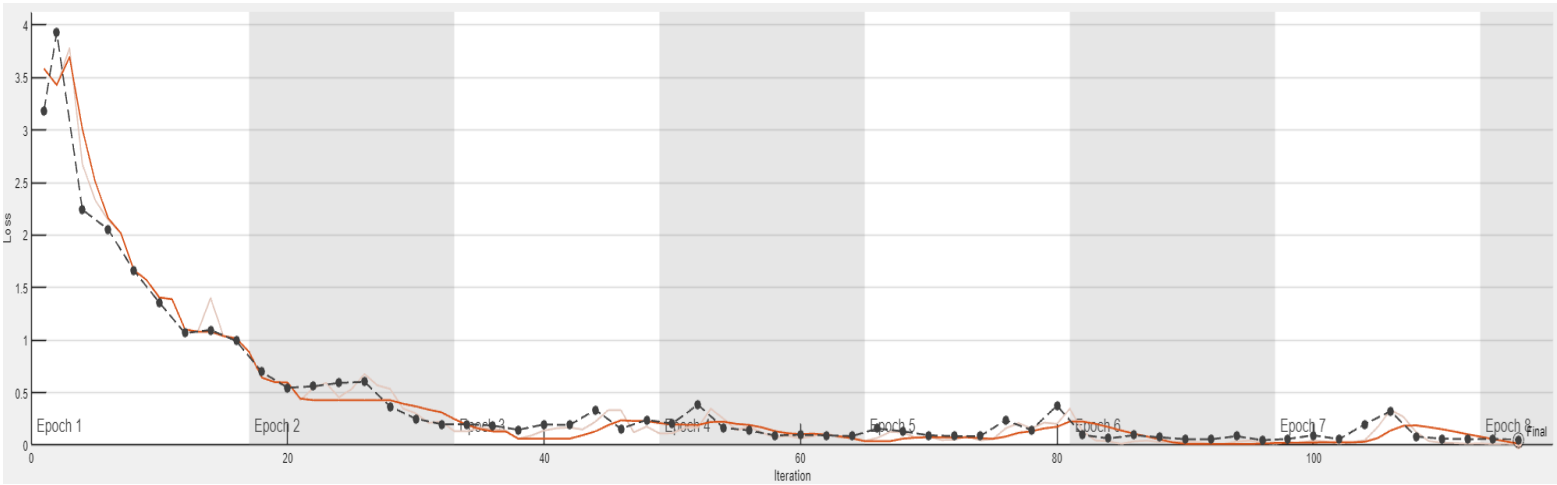
Na podstawie przeprowadzonych eksperymentów oraz analizy uzyskanych wyników wybrano najbardziej optymalną konfigurację sieci neuronowej. Model ten składa się z trzech warstw i jest trenowany z wykorzystaniem algorytmu optymalizacji RMSProp za pomocą niezaszumionych danych.

Tabela 6. Wyniki osiągnięte przez optymalny model

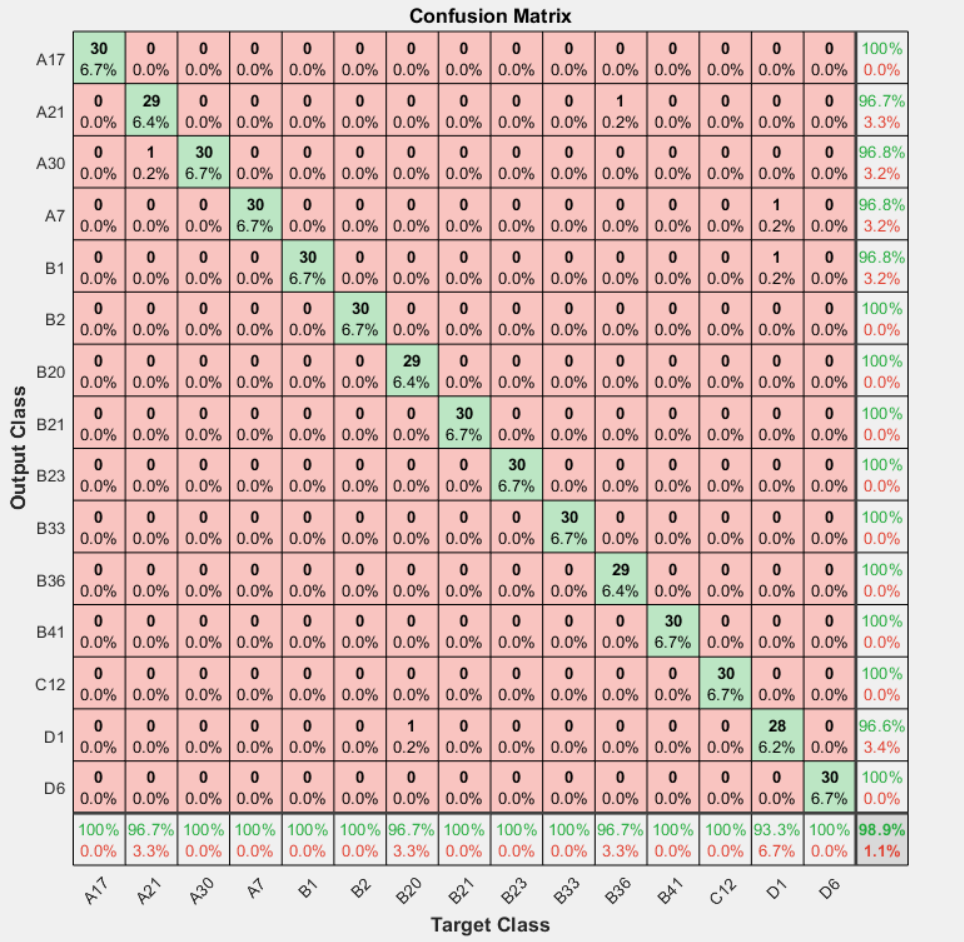
| Warunki | | Optymalny model | | | | |
|---|---|---|---|---|---|---|
| czas treningu | | 6s | | | | |
| liczba epok | | 8 | | | | |
| liczba iteracji | | 116 | | | | |
| klasa | | Precyzja | Czułość | Swoistość | F1 | Dokładność |
| 1 | A17 | 100% | 100% | 100% | - | - |
| 2 | A21 | 96,67% | 96,67% | 99,76% | - | - |
| 3 | A30 | 96,77% | 100% | 99,76% | - | - |
| 4 | A7 | 96,77% | 100% | 99,76% | - | - |
| 5 | B1 | 96,77% | 100% | 99,76% | - | - |
| 6 | B2 | 100% | 100% | 100% | - | - |
| 7 | B20 | 100% | 96,67% | 100% | - | - |
| 8 | B21 | 100% | 100% | 100% | - | - |
| 9 | B23 | 100% | 100% | 100% | - | - |
| 10 | B33 | 100% | 100% | 100% | - | - |
| 11 | B36 | 100% | 96,67% | 100% | - | - |
| 12 | B41 | 100% | 100% | 100% | - | - |
| 13 | C12 | 100% | 100% | 100% | - | - |
| 14 | D1 | 96,55% | 93,33% | 99,76% | - | - |
| 15 | D6 | 100% | 100% | 100% | - | - |
| średnia | | 98,90% | 98,89% | 99,92% | 98,90% | 98,89% |




Rys 4. Przebieg funkcji dokładności optymalnego modelu




Rys 5. Przebieg funkcji strat optymalnego modelu




Rys 6. Macierz pomyłek optymalnego modelu

Optymalny model zgodnie z oczekiwaniami, osiąga bardzo wysoką skuteczność klasyfikacji. Pojedyncze błędy miały charakter przypadkowy. Analiza macierzy pomyłek pozwala zidentyfikować nieliczne klasy, które były mylone. Najgorszy wynik sieć uzyskała dla klasy D1, która została pomylona dwa razy.

## 6. Podsumowanie

Wybrany optymalny model spełnił wszystkie założenia projektowe, osiągając wysoką dokładność przy zachowaniu efektywności czasowej i obliczeniowej. Wyniki potwierdzają, że optymalne rozwiązanie nadaje się do praktycznego zastosowania w systemach wspomagania kierowców lub autonomicznych samochodach. Przeprowadzone eksperymenty dostarczyły cennych wskazówek dotyczących optymalizacji podobnych modeli w przyszłości.